



UNIVERSIDADE FEDERAL DE GOIÁS
INSTITUTO DE INFORMÁTICA
ESPECIALIZAÇÃO EM RESIDÊNCIA EM TECNOLOGIA DA INFORMAÇÃO

SAMUEL VICTOR OLIVEIRA LIMA

**NOTIFY FLOW: RELATÓRIO TÉCNICO FINAL DE UMA
PLATAFORMA MULTICANAL DE NOTIFICAÇÕES
CONSENTIDAS**

Relatório Técnico Final — Gate 5

Goiânia
2026
UNIVERSIDADE FEDERAL DE GOIÁS
INSTITUTO DE INFORMÁTICA
ESPECIALIZAÇÃO EM RESIDÊNCIA EM TECNOLOGIA DA INFORMAÇÃO

SAMUEL VICTOR OLIVEIRA LIMA

**NOTIFY FLOW: RELATÓRIO TÉCNICO FINAL DE UMA
PLATAFORMA MULTICANAL DE NOTIFICAÇÕES
CONSENTIDAS**

Relatório Técnico Final apresentado ao Programa de Pós-Graduação Lato Sensu Especialização em Residência em Tecnologia da Informação do Instituto de Informática da Universidade Federal de Goiás, como requisito parcial para avaliação final do Gate 5 do Projeto de Aplicação.

Orientador: Prof. Leonardo Oliveira

RESUMO

Este relatório técnico final apresenta o Notify Flow, produto de software desenvolvido na modalidade Projeto de Aplicação para centralizar notificações consentidas por WhatsApp Cloud API oficial, Telegram Bot API e Gmail. O problema tratado é a fragmentação entre canais, a dificuldade de comprovar autorização e a exposição operacional causada por disparos sem observar as regras dos provedores. A solução reúne credenciais, contatos, convites, grupos, templates, conjuntos multicanais, campanhas, conversas, eventos de webhook e trilhas de auditoria, preservando autorização independente por canal.

A validação final combinou inspeção arquitetural, testes automatizados de unidade e contrato, análise estática, build de produção, revisão da configuração de contêineres e ensaio funcional. Em 14 de agosto de 2026, a API concluiu 374 testes e o frontend 244, totalizando 618 testes aprovados. O lint da API e o build Vite também foram aprovados. Além disso, foi observado pelo autor um envio em massa com aproximadamente 20 a 30 usuários, sem interrupção global quando ocorreram falhas individuais e com logs coerentes por contato e canal. Esse ensaio é evidência funcional, não benchmark de desempenho nem estudo estatístico.

Os resultados mostram um produto web final demonstrável e reproduzível por Docker, com fila BullMQ/Redis, estado durável no MongoDB, mídia em GridFS, atualização administrativa por Socket.IO e integração oficial com a Meta. Cada entrega mantém estado, tentativas, motivo e recibos próprios, o que evita que uma falha individual interrompa toda a campanha. A contribuição central é combinar mensageria oficial, consentimento rastreável e autogestão do titular em uma única interface. A solução oferece controles que apoiam a LGPD, mas não substitui governança, base legal, política institucional ou avaliação jurídica.

Palavras-chave: notificações multicanal; WhatsApp Cloud API; Telegram Bot API; consentimento; LGPD; BullMQ; Redis; MongoDB; webhooks; observabilidade; whitelabel.

LISTA DE FIGURAS

Figura 1 — Arquitetura ponta a ponta do Notify Flow.....	10
Figura 2 — Jornada de consentimento e vínculo de identidades.....	12
Figura 3 — Área pública Meu Perfil em desktop e dispositivo móvel.....	15
Figura 4 — Preparação do WhatsApp Cloud para produção.....	16
Figura 5 — Ciclo de entrega confiável, tentativas e auditoria.....	20

LISTA DE QUADROS E TABELAS

Quadro 1 — Perfis e responsabilidades.....	8
Quadro 2 — Componentes da arquitetura.....	11
Quadro 3 — Regras de autorização por canal.....	12
Quadro 4 — Requisitos para produção no WhatsApp Cloud.....	17
Quadro 5 — Variáveis de ambiente por domínio.....	22
Tabela 1 — Evolução do Gate 4 para o Gate 5.....	8
Tabela 2 — Evidências de testes e validações.....	24
Tabela 3 — Riscos, controles e limitações.....	26
Tabela 4 — Histórias de usuário do administrador.....	31
Tabela 5 — Histórias de usuário do contato.....	32
Tabela 6 — Matriz de rastreabilidade dos requisitos finais.....	34

LISTA DE SIGLAS E ABREVIATURAS

Sigla	Significado
API	Interface de Programação de Aplicações
CNPJ	Cadastro Nacional da Pessoa Jurídica
DTO	Objeto de Transferência de Dados
HMAC	Código de Autenticação de Mensagem com Hash
JWT	JSON Web Token
LGPD	Lei Geral de Proteção de Dados Pessoais
MVP	Produto Mínimo Viável, utilizado como linha de base histórica
PII	Informação Pessoal Identificável
SMTP	Protocolo Simples de Transferência de Correio
TTL	Tempo de vida de um registro ou chave
WABA	Conta do WhatsApp Business
WSS	WebSocket Seguro

SUMÁRIO

1 INTRODUÇÃO.....	1
1.1 Contexto e problema.....	1
1.2 Objetivo geral e objetivos específicos.....	1
1.3 Escopo final do produto.....	2
1.4 Relevância e impacto esperado.....	2
2 FUNDAMENTAÇÃO TÉCNICA E NORMATIVA.....	4
2.1 Mensageria oficial, janela de atendimento e modelos.....	4
2.2 Consentimento, minimização e direitos do titular.....	4
2.3 Processamento assíncrono e observabilidade.....	5
2.4 Contêineres e infraestrutura como código.....	6
3 METODOLOGIA E EVOLUÇÃO STAGE-GATE.....	7
3.1 Estratégia de desenvolvimento.....	7
3.2 Evolução dos Gates 1 a 5.....	7
3.3 Perfis e histórias de usuário.....	8
3.4 Estratégia de testes e critérios de aceitação.....	8
3.5 Limitações metodológicas.....	9
4 DESENVOLVIMENTO, RESULTADOS E DISCUSSÃO.....	10
4.1 Visão geral da solução.....	10
4.2 Fluxo orientado ao consentimento.....	11
4.3 Funcionalidades do administrador.....	13
4.4 Funcionalidades do contato e Meu Perfil.....	14
4.5 Integração com WhatsApp Cloud e Meta.....	15
4.6 Integração com Telegram.....	18
4.7 Integração com Gmail.....	19
4.8 Fila, tentativas, recibos e logs.....	19
4.9 Execução local com Docker.....	20
4.10 Implantação no Render por Blueprint.....	21
4.11 Whitelabel, mídia e administração do armazenamento.....	22
4.12 Webhooks, conversas, alertas e tempo real.....	23
5 VALIDAÇÃO E RESULTADOS FINAIS.....	24
5.1 Ambiente, procedimento e evidências.....	24
5.2 Ensaio de envio em massa e integrações.....	24
5.3 Resultados por requisito e impacto.....	25
6 ENTREGA, OPERAÇÃO E MANUTENÇÃO.....	27
6.1 Manifesto do pacote Gate 5.....	27
6.2 Execução e verificação operacional.....	27
6.3 Empacotamento seguro e retenção.....	27
6.4 Roteiro de aceite do produto.....	28
7 CONSIDERAÇÕES FINAIS.....	29
REFERÊNCIAS.....	30
APÊNDICE A — HISTÓRIAS DE USUÁRIO.....	31
APÊNDICE B — CHECKLIST DE CONFIGURAÇÃO.....	33
APÊNDICE C — MATRIZ DE EVIDÊNCIAS.....	34
APÊNDICE D — COMANDOS DE VALIDAÇÃO E EMPACOTAMENTO.....	35
APÊNDICE E — ORGANIZAÇÃO DAS EVIDÊNCIAS.....	36
APÊNDICE F — EVOLUÇÃO DOCUMENTAL.....	37

Nota de compatibilidade

O arquivo foi preparado em DOCX para edição no Microsoft Word e importação no Google Docs. O sumário foi materializado para permanecer visível também no PDF; caso o conteúdo seja alterado, revise suas páginas antes de gerar uma nova versão. A versão PDF incluída na entrega registra a paginação final verificada.

1 INTRODUÇÃO

1.1 Contexto e problema

Organizações que precisam avisar clientes, servidores ou participantes de programas frequentemente combinam mensageria instantânea e e-mail. Na prática, cada provedor possui identificadores, políticas, limites, modelos e formas de comprovar entrega diferentes. Quando esses detalhes são tratados por planilhas, scripts isolados ou sessões não oficiais, o administrador perde a visão de quem autorizou cada canal, por qual campanha a pessoa foi incluída e por que uma entrega falhou.

O WhatsApp concentra a dor mais sensível. Fora da janela de atendimento iniciada pelo cliente, a organização deve utilizar modelos aprovados na conta remetente. A automação de uma sessão comum do WhatsApp Web não equivale à API oficial e, quando empregada para disparos, amplia o risco de restrição do número e não oferece as mesmas garantias de governança. O Notify Flow foi concebido para trabalhar com o WhatsApp Cloud API e deixar a conversa livre restrita à janela permitida, mantendo o Telegram e o e-mail como canais complementares.

O segundo problema é o consentimento. A simples existência de um telefone, endereço de e-mail ou nome de usuário não prova que a pessoa deseja receber notificações. O sistema precisa capturar a interação real, vincular identidades sem duplicar contatos, registrar a origem da autorização, possibilitar revogação e impedir que uma entrega não autorizada seja silenciosamente tratada como sucesso.

1.2 Objetivo geral e objetivos específicos

O objetivo geral é demonstrar a viabilidade de uma plataforma web responsiva que centraliza notificações multicanal consentidas, com filas resilientes, logs auditáveis, integrações oficiais e recursos de autogestão para o contato.

- separar as configurações e permissões de WhatsApp Cloud, Telegram e e-mail;
- permitir a criação amigável de templates por canal e de conjuntos multicanais;
- registrar contatos por interação, convite ou cadastro administrativo sem criar destinos artificiais;

- processar campanhas em fila, isolando a falha de um destinatário dos demais;
- exibir mensagens, recibos, tentativas e erros em tempo real sem expor segredos;
- oferecer ao contato uma área segura para consultar dados, vínculos e histórico;
- documentar a execução local em Docker e a implantação por Blueprint no Render;
- validar o produto final por testes automatizados e critérios de aceitação reproduzíveis.

1.3 Escopo final do produto

O escopo abrange painel administrativo, página pública de convite, área Meu Perfil, API REST, comunicação em tempo real, persistência MongoDB, fila BullMQ sobre Redis, webhooks do WhatsApp Cloud e Telegram, SMTP do Gmail, Docker Compose e Blueprint do Render. O produto não pretende substituir uma central completa de atendimento, um provedor de CRM ou a governança nativa da Meta. Também não aprova modelos no WhatsApp Manager: o administrador cadastra no Notify Flow o nome e a estrutura de um modelo que já existe na conta remetente.

A solução atual não usa WhatsApp Web JS para iniciar notificações. As conversas exibidas no painel WhatsApp são constituídas pelos eventos recebidos no webhook oficial e pelas respostas enviadas pela Graph API. A plataforma também não importa um histórico completo da Meta; ela mantém o histórico que efetivamente atravessa sua própria integração.

O produto também oferece personalização whitelabel por instalação, armazenamento de mídias em GridFS, exportação sanitizada por collection, limpeza administrativa auditada e alertas opcionais por Gmail quando chegam mensagens pelos webhooks. Não há multi-tenancy, aplicativo móvel nativo, executável Electron ou garantia de importação/restauração integral do banco. A forma executável entregue é a aplicação web containerizada e reproduzível.

1.4 Relevância e impacto esperado

O valor do projeto não está apenas em enviar mensagens. Está em tornar explícito o caminho entre consentimento, identidade, template, fila, provedor e recibo. Esse encadeamento permite que o administrador explique por que determinado contato recebeu

— ou deixou de receber — uma comunicação, ao mesmo tempo em que oferece ao titular mecanismos de consulta e revogação.

Hipótese de impacto

Ao concentrar notificações na API oficial do WhatsApp e registrar consentimento por canal, o Notify Flow reduz risco de bloqueio por automação inadequada, retrabalho operacional e envios indevidos; aumenta, ainda, a rastreabilidade de campanhas e a autonomia do contato.

2 FUNDAMENTAÇÃO TÉCNICA E NORMATIVA

2.1 Mensageria oficial, janela de atendimento e modelos

A WhatsApp Business Platform diferencia mensagens de serviço, enviadas dentro de uma janela de atendimento iniciada pelo usuário, e mensagens iniciadas pela empresa. A janela dura 24 horas e é renovada quando o cliente envia uma nova mensagem. Nesse período, respostas livres de serviço não são cobradas segundo a página de preços consultada; fora dele, a empresa deve usar um modelo aprovado e está sujeita à cobrança por mensagem entregue, de acordo com mercado e categoria. As categorias atuais incluem marketing, utilidade, autenticação e serviço (META, 2026a).

O modelo aprovado é um contrato entre a conta remetente e a Meta. Seu nome, idioma, componentes e parâmetros precisam corresponder ao artefato cadastrado no WhatsApp Manager. Por isso, o Notify Flow oferece formulários que montam o payload sem exigir JSON do usuário, mas não presume aprovação. Os itens de exemplo vinculados ao número de teste ou ao número de produção só funcionam quando o modelo homônimo está disponível para aquele remetente.

No Telegram, o bot pode responder a pessoas que iniciaram ou autorizaram a conversa. Ele não descobre o telefone arbitrariamente: o número só é confiável quando o próprio usuário compartilha o contato e o identificador do contato corresponde ao remetente. Já o e-mail utiliza SMTP autenticado e exige endereço válido, permissão ativa e, nos fluxos de atualização pelo chat, confirmação por código temporário.

2.2 Consentimento, minimização e direitos do titular

A LGPD exige finalidade, adequação, necessidade, transparência, segurança e mecanismos para o exercício de direitos. No contexto do produto, isso se traduz em autorizações independentes, termos apresentados na jornada pública, registro da fonte da decisão e proibição de inventar uma identidade de destino apenas porque outro canal foi autorizado. O contato pode consultar seus dados e entregas, revogar um canal, remover vínculos próprios com convites e sair de grupos mediante confirmação.

Os dados sensíveis de configuração são criptografados com AES-256-GCM e os identificadores pesquisáveis usam índices cegos baseados em HMAC. A exclusão do contato pseudonimiza registros históricos necessários à auditoria, em vez de prometer apagamento indiscriminado. Essa distinção é importante: a plataforma reduz exposição e preserva comprovação operacional, mas não implementa criptografia de ponta a ponta entre o administrador e os provedores.

2.3 Processamento assíncrono e observabilidade

Uma campanha multicanal é naturalmente sujeita a falhas parciais. O endereço pode estar inválido, o modelo pode não existir na conta, a janela do WhatsApp pode estar fechada, um token pode expirar ou o provedor pode responder com limitação temporária. Processar todos os destinatários na mesma requisição HTTP prolongaria o tempo de resposta e permitiria que uma falha interrompesse o conjunto.

BullMQ materializa cada entrega como um trabalho em Redis. O worker pode operar com concorrência controlada, repetir falhas transitórias e preservar o estado do restante da campanha. No Notify Flow, o worker usa concorrência cinco, até quatro tentativas e backoff exponencial a partir de dois segundos. O MongoDB mantém o registro durável da entrega e um processo de recuperação procura itens pendentes de enfileiramento. A semântica externa deve ser descrita como pelo menos uma vez em cenários ambíguos: idempotência e travas reduzem duplicidade local, mas nenhum sistema distribuído deve prometer exatamente uma entrega quando o provedor aceitou a mensagem e a resposta se perdeu.

Observabilidade significa correlacionar requisição, campanha, entrega e recibo sem registrar segredos. A API atribui requestId, redige campos sensíveis e mantém resultados por contato e canal. WebSockets transportam atualizações incrementais para o painel; webhooks transportam eventos dos provedores para a API. São mecanismos complementares: webhook é servidor a servidor; WebSocket mantém a interface conectada ao backend.

2.4 Contêineres e infraestrutura como código

O Docker Compose descreve um ambiente local composto por MongoDB, Redis, API e frontend Nginx. O Blueprint do Render cumpre papel semelhante em produção, mas não replica o Compose literalmente: o frontend é um serviço web público, a API é um serviço privado e o Redis é um Key Value gerenciado. O MongoDB é fornecido externamente pelo Atlas. Variáveis não sensíveis podem ser declaradas no YAML; segredos devem ser gerados ou solicitados durante a sincronização, nunca versionados.

3 METODOLOGIA E EVOLUÇÃO STAGE-GATE

3.1 Estratégia de desenvolvimento

O trabalho foi desenvolvido individualmente por Samuel Victor Oliveira Lima, com orientação de Leonardo Oliveira, por ciclos incrementais. Cada ciclo partiu de um fluxo observável — por exemplo, receber um webhook, autorizar um canal, criar uma campanha ou abrir o Meu Perfil — e foi refinado até que regras de negócio, interface, persistência e testes convergissem. O repositório separa `api` e `frontend`, possui documentação própria em cada diretório e mantém a infraestrutura local e de produção na raiz.

A análise final utilizou quatro fontes de evidência: inspeção do código e dos artefatos de infraestrutura; execução das suítes automatizadas; build e análise estática; e testes manuais conduzidos pelo autor nos painéis de Meta, Telegram e Gmail durante a implementação. Nenhum dado de credencial foi reproduzido neste documento.

3.2 Evolução dos Gates 1 a 5

O processo Stage-Gate permitiu reduzir incertezas em etapas. Os primeiros Gates delimitaram problema, público e viabilidade; as etapas intermediárias consolidaram arquitetura, integrações e demonstração do MVP; o Gate 4 formalizou impacto, testes e validação; e o Gate 5 reúne o produto técnico, a documentação final, as evidências atualizadas e o plano de operação. Essa evolução evita apresentar a versão final como uma reescrita desconectada das decisões anteriores.

Gate	Foco predominante	Resultado incorporado ao produto final
1	Problema e oportunidade	Necessidade de mensageria multicanal com consentimento.
2	Requisitos e solução	Perfis, jornadas, canais e arquitetura inicial.
3	Construção e integração	API, SPA, persistência, webhooks e filas.
4	Impacto, testes e validação	Linha de base de 514 testes e ensaio funcional.
5	Entrega final	618 testes, operação documentada, whitelabel, storage e pacote técnico.

Tabela 1 — Evolução do Gate 4 para o Gate 5

Fonte: Elaboração própria (2026).

3.3 Perfis e histórias de usuário

Perfil	Responsabilidade	Resultado esperado
Administrador	Configurar canais, contatos, templates, convites, campanhas e documentos.	Operar envios com rastreabilidade, sem que uma falha paralise os demais.
Contato	Iniciar a interação, consentir, consultar dados, vínculos e entregas.	Controlar seus canais e receber apenas comunicações autorizadas.
Provedor	Validar credenciais, aceitar mensagens e devolver eventos/recibos.	Aplicar políticas próprias; o Notify Flow não as contorna.

Quadro 1 — Perfis e responsabilidades

Fonte: Elaboração própria (2026).

As histórias completas estão no Apêndice A. Elas foram convertidas em critérios verificáveis, como impedir resposta livre fora da janela de 24 horas, ignorar um destinatário sem consentimento sem abortar a campanha e exigir confirmação para revogar vínculo ou permissão.

3.4 Estratégia de testes e critérios de aceitação

A API foi testada com o test runner nativo do Node, isolando managers, controllers, middlewares, validações, criptografia, filas, webhooks e políticas de consentimento. O frontend foi testado com Vitest, cobrindo serviços HTTP, autenticação, sockets, contatos, convites, perfil, templates, campanhas, diálogos e navegação. O lint verificou consistência da API; o build Vite demonstrou que os módulos do frontend são empacotáveis para produção; e `docker compose config --quiet` validou a composição estática.

todas as suítes automatizadas devem encerrar sem falha;
nenhum segredo real deve aparecer no código, nos logs ou no relatório;
o webhook do WhatsApp deve validar challenge e assinatura HMAC;
o webhook do Telegram deve validar segredo e deduplicar update_id;
a campanha deve continuar quando um destinatário falha ou não autorizou o canal;
a resposta livre do WhatsApp deve respeitar a janela de 24 horas;
o contato deve poder revogar e consultar as próprias permissões;
a versão de produção do frontend deve ser compilada com sucesso.

3.5 Limitações metodológicas

Os testes automatizados são predominantemente unitários e de contrato, com dependências externas simuladas. Eles não substituem um teste ponta a ponta contínuo contra Atlas, Redis, SMTP, Telegram, Graph API e Render. Também não foi realizada pesquisa de usabilidade com participantes externos nem comparação estatística antes/depois. Assim, o impacto é analisado por evidência funcional, redução de risco arquitetural e capacidade de auditoria; não como ganho quantitativo já comprovado em uma organização.

4 DESENVOLVIMENTO, RESULTADOS E DISCUSSÃO

4.1 Visão geral da solução

O Notify Flow é uma aplicação web responsiva composta por três superfícies. A primeira é o painel administrativo, no qual se configuram canais, contatos, templates, conjuntos, convites, campanhas e documentos legais. A segunda é a camada pública de convite, que apresenta termos e direciona o visitante ao canal selecionado. A terceira é o Meu Perfil, área em que o contato consulta seus dados, permissões, convites, grupos e histórico de entregas.

Arquitetura ponta a ponta

Separação em camadas, integrações oficiais, processamento assíncrono e auditoria

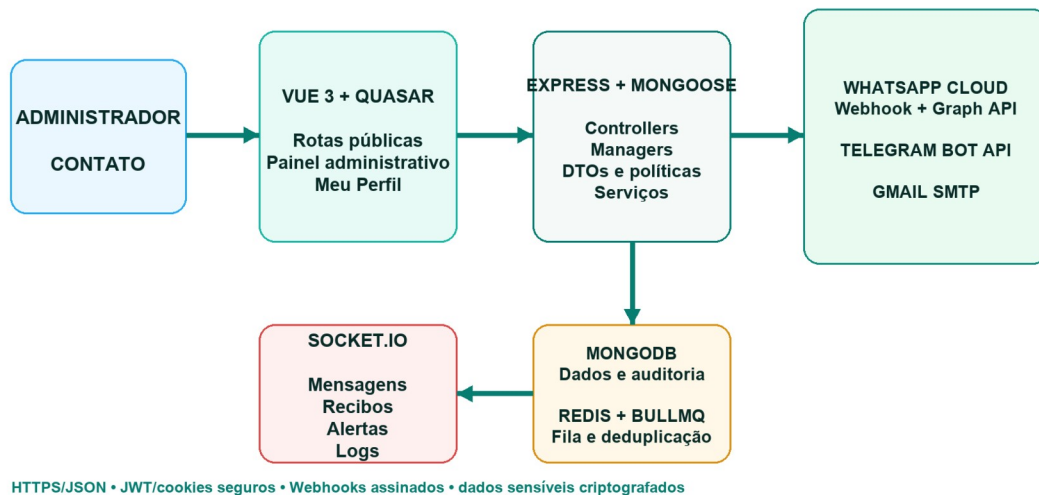


Figura 1 — Arquitetura ponta a ponta do Notify Flow

Fonte: Elaboração própria (2026).

Componente	Tecnologia	Responsabilidade principal
Frontend	Vue 3, Quasar, Pinia, Vue Router, Axios	Interface responsiva, sessões separadas, preview e administração.
Tempo real	Socket.IO	Eventos, mensagens, recibos, logs e central administrativa.
API	Node.js, Express, Zod	Rotas, validação, autorização, políticas e orquestração.
Dados	MongoDB, Mongoose	Contatos, consentimentos, campanhas, eventos e configurações criptografadas.

Componente	Tecnologia	Responsabilidade principal
Fila	Redis/Valkey, BullMQ	Concorrência, tentativas, backoff e recuperação de entregas.
Provedores	Meta Graph API, Telegram Bot API, Gmail SMTP	Entrega e recepção de eventos externos.
Infraestrutura	Docker Compose, Nginx, Render Blueprint	Execução local e implantação reproduzível.

Quadro 2 — Componentes da arquitetura

Fonte: Elaboração própria (2026).

4.2 Fluxo orientado ao consentimento

O contato pode chegar por convite, comando ou cadastro administrativo. Um clique de convite gera rastreabilidade, mas não autoriza sozinho o destino: a identidade real precisa ser observada pelo webhook ou confirmada por um mecanismo seguro. O WhatsApp e o Telegram reconhecem comandos configuráveis; o e-mail digitado em qualquer momento do chat inicia um desafio de validação com seis dígitos, válido por quinze minutos e reenviável após dois minutos.

Jornada de consentimento e vínculo

O contato inicia a interação; cada canal mantém autorização independente e revogável

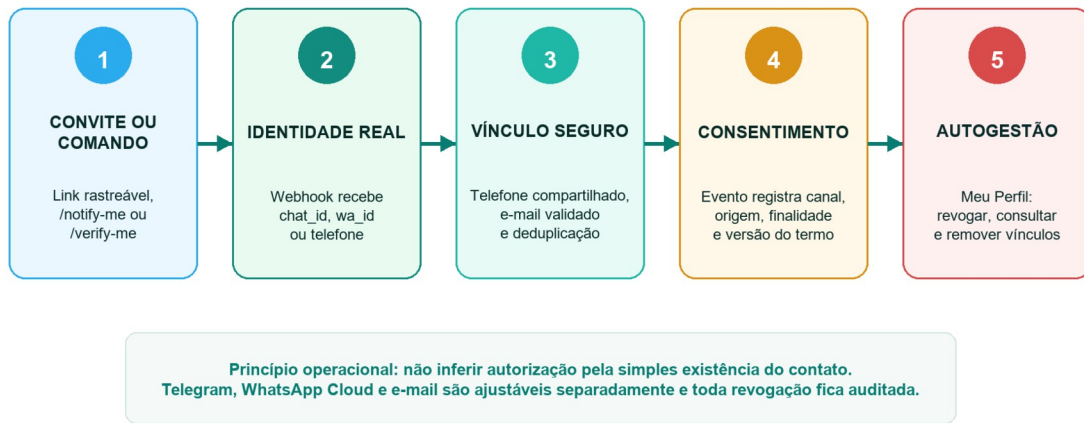


Figura 2 — Jornada de consentimento e vínculo de identidades

Fonte: Elaboração própria (2026).

Canal	Como a identidade é confirmada	Como autoriza	Como revoga
WhatsApp Cloud	wa_id/telefone observado no webhook oficial.	Comando dinâmico, Meu Perfil ou decisão administrativa confirmada.	Meu Perfil ou painel administrativo com confirmação.
Telegram	chat_id recebido; telefone apenas por contact.user_id igual ao remetente.	Comando/invite assinado e interação com o bot.	Comando de parada, Meu Perfil ou administração.
E-mail	Código enviado ao endereço informado e validado no chat/perfil.	Ativado após validação ou confirmação no Meu Perfil.	Meu Perfil ou administração.

Quadro 3 — Regras de autorização por canal

Fonte: Elaboração própria (2026).

O vínculo entre Telegram e WhatsApp só é consolidado quando existe evidência comum, como o telefone compartilhado pelo próprio usuário. O sistema consulta índices cegos para localizar um contato existente, transfere a identidade e evita criar perfis paralelos. Se outro canal ainda não possui identidade real, a autorização pode permanecer pendente, mas o sistema não fabrica um chat_id ou wa_id.

4.3 Funcionalidades do administrador

4.3.1 Início e configurações

A tela inicial salva cada provedor separadamente. O WhatsApp recebe token, Phone Number ID, Business Account ID, número público, versão da Graph API, App Secret e verify token; o Telegram recebe token e URL pública, identifica o bot por getMe e registra o webhook; o Gmail recebe conta, senha de aplicativo e remetente. Os campos persistidos aparecem mascarados e só podem ser revelados por administrador autenticado, em rota sem cache, limitada e auditada. Também são configuráveis os comandos, respostas amigáveis, links úteis e textos de autorização.

4.3.2 Contatos, convites e grupos

O administrador busca, cadastra, edita e pseudonimiza contatos, acompanha identidades técnicas e permissões e cria grupos. Convites possuem slug, identidade visual, QR Code e links por canal. Quando o contato chega por um convite, o vínculo é armazenado; a sincronização por convite cria ou atualiza grupos sem duplicar participantes. Um mesmo contato pode ter mais de um convite e, por isso, participar de mais de um segmento.

4.3.3 Templates e conjuntos multicanais

A biblioteca oferece editores específicos. WhatsApp usa nome oficial, idioma, componentes e parâmetros; Telegram suporta texto, mídia validada e menus; e-mail suporta texto ou HTML sanitizado. Um conjunto agrega de um a três templates, no máximo um por canal, e pode ser associado a um convite. O mesmo template pode participar de vários conjuntos. Essa modelagem reduz divergência entre campanhas recorrentes e evita que o administrador manipule payloads JSON manualmente.

4.3.4 Notificações, canais e governança

A campanha pode usar um conjunto, templates isolados ou uma mensagem rápida quando a política do canal permite. O administrador escolhe um, dois ou três canais, contatos ou grupos, preenche variáveis e revisa uma prévia antes de confirmar. As páginas

de WhatsApp, Telegram e Gmail reúnem disparos específicos; WhatsApp e Telegram também mostram conversas em tempo real. Governança inclui termos e privacidade versionados, auditoria de links temporários e central de eventos administrativos com retenção de trinta dias.

4.4 Funcionalidades do contato e Meu Perfil

O acesso do contato é separado da sessão administrativa. O link temporário é assinado, de uso único e possui validade configurável de até sete dias; desafios de código possuem tentativas e limites de reenvio. Os comandos `/login` e `/meu-perfil` podem gerar o link no WhatsApp ou Telegram. No perfil, o contato edita dados, ativa ou revoga e-mail, consulta identidades dos canais, remove seus próprios vínculos com convites, sai de grupos e acompanha apenas as entregas destinadas a ele.

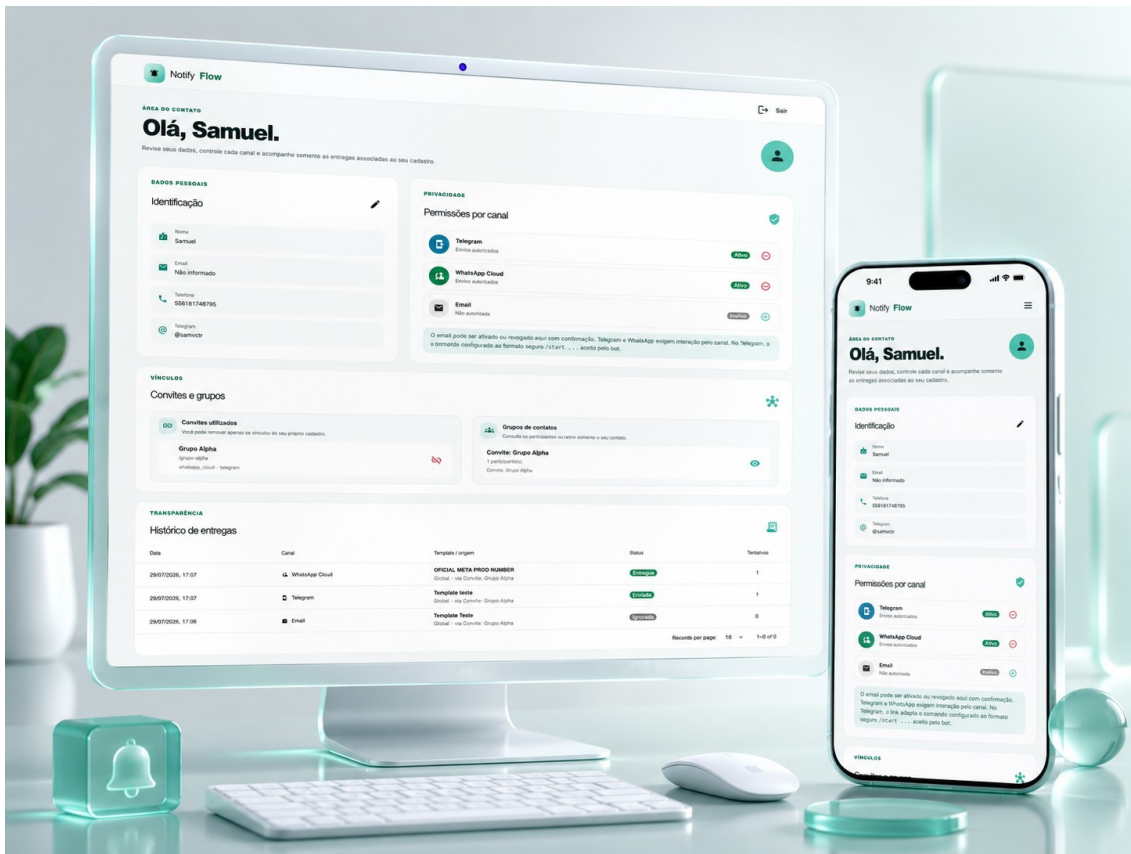


Figura 3 — Área pública Meu Perfil em desktop e dispositivo móvel

Fonte: Arte visual do projeto Notify Flow (2026).

A separação de sessões limita a superfície de privilégio: o link do contato não concede acesso ao painel administrativo. Da mesma forma, desativar um canal não elimina automaticamente o histórico já pseudonimizado, mas impede novas entregas naquele destino. A interface explicita convites e grupos para que o titular compreenda por que pode ser selecionado em uma campanha.

4.5 Integração com WhatsApp Cloud e Meta

4.5.1 Preparação empresarial e burocrática

A implantação produtiva começa fora do código. A organização precisa de empresa real com dados consistentes, conta do Facebook protegida por autenticação em dois fatores, Página e portfólio empresarial no Business Manager/Meta Business Suite. A Meta pode solicitar atos constitutivos, licença, documento fiscal ou conta de serviço, todos legíveis,

válidos e compatíveis com o nome, endereço e telefone informados. Não existe um prazo universal de aprovação; portanto, a verificação deve ser tratada como dependência de cronograma, e não como atividade concluída no dia da entrega.

Para reduzir risco operacional, recomenda-se um número dedicado, novo ou confirmado como elegível para o fluxo disponível na conta. A afirmação de que todo número precisa ser absolutamente inédito seria excessiva, pois migração e coexistência dependem dos recursos oferecidos pela Meta; entretanto, reutilizar um número já preso a outra conta sem preparar a migração é uma causa comum de falha de registro. O painel da conta é a autoridade sobre elegibilidade.

Preparação do WhatsApp Cloud para produção

A sequência combina requisitos empresariais, segurança, credenciais e homologação de modelos

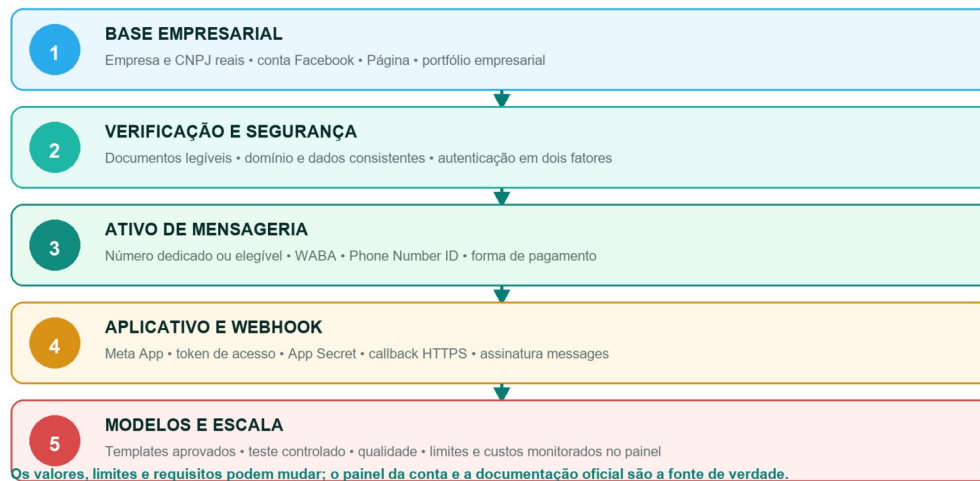


Figura 4 — Preparação do WhatsApp Cloud para produção

Fonte: Elaboração própria (2026).

Etapa	Evidência necessária	Risco se omitida
Empresa e portfólio	CNPJ e dados públicos consistentes.	Bloqueio da verificação ou limitação do portfólio.
Segurança	2FA, administradores legítimos e acesso mínimo.	Tomada de conta ou rejeição de ativos.
Número	Recebimento de SMS/ligação e elegibilidade.	Impossibilidade de registrar o remetente.
Aplicativo	Meta App, produto WhatsApp e permissões.	Token ou webhook sem acesso ao WABA.
Webhook	URL HTTPS, verify token e App Secret.	Eventos rejeitados ou sem autenticidade.

Etapa	Evidência necessária	Risco se omitida
Pagamento	Forma válida para mensagens cobradas.	Bloqueio de mensagens iniciadas pela empresa.
Modelos	Nome/idioma aprovados na conta remetente.	Erro de template inexistente ou não aprovado.

Quadro 4 — Requisitos para produção no WhatsApp Cloud

Fonte: Elaboração própria (2026).

4.5.2 Credenciais, webhook e comandos

O Phone Number ID é o identificador técnico usado na rota `/messages`; o Business Account ID identifica a WABA; o access token autoriza a Graph API; o App Secret valida a assinatura `X-Hub-Signature-256`; e o verify token é um segredo escolhido pelo administrador para o challenge inicial do webhook. O número público é mantido separadamente porque serve a links `wa.me` e à apresentação ao usuário. Em produção, tokens devem ser permanentes ou renovados por processo seguro, nunca colados em repositórios ou documentos.

Ao receber `/notify-me`, o fluxo registra o consentimento WhatsApp para identidades reais disponíveis e devolve instruções amigáveis. `/login` gera um link temporário de acesso ao Meu Perfil. Um e-mail válido digitado durante a conversa inicia sua validação; somente após o código correto o endereço é criado ou alterado e a permissão de e-mail é ativada. A resposta livre continua limitada à janela de 24 horas; a notificação iniciada pela organização usa modelo aprovado.

A instalação mantém três exemplos protegidos contra exclusão: `jaspers_market_plain_text_v1` e `jaspers_market_order_confirmation_v1`, associados ao ambiente de número de teste da Meta, e `3p_direct_integration_test_template`, usado para validar a integração do número de produção em modo de teste. Eles não são modelos universais: o mesmo nome e idioma precisam existir e estar aprovados na WABA que atende o remetente. O antigo `verify_code_1` foi retirado da lista fixa; o login atual usa o comando `/login` e um link temporário assinado, sem depender desse modelo de autenticação.

4.5.3 Custos, qualidade e limites

Na política consultada em julho de 2026, a cobrança da WhatsApp Business Platform ocorre por mensagem entregue e varia por mercado e categoria. Mensagens de serviço não são cobradas e mensagens de utilidade enviadas em resposta ao usuário dentro da janela de atendimento também podem ser gratuitas. Marketing, autenticação e utilidade fora da janela seguem as tabelas vigentes. Como preços e regras mudam, o relatório não fixa um valor em reais: a estimativa deve usar a página oficial e o país do destinatário no momento da contratação (META, 2026a).

A escala de mensagens iniciadas pela empresa depende do nível exibido no WhatsApp Manager, da verificação e da qualidade. O material oficial de onboarding descreve evolução por degraus como 1 mil, 10 mil, 100 mil destinatários únicos por período de 24 horas e, depois, ilimitado, mas contas de teste ou novas podem apresentar limites iniciais distintos. O painel da própria conta, e não um número reproduzido em documentação acadêmica, é a fonte de verdade. Bloqueios, denúncias e baixa qualidade podem impedir a expansão.

Discussão sobre a principal dor

A API oficial não elimina toda possibilidade de restrição, porque políticas e qualidade ainda se aplicam. Ela, porém, fornece o caminho suportado para modelos, limites, recibos e cobrança. Isso é qualitativamente mais seguro e auditável do que automatizar uma sessão WhatsApp Web para disparos iniciados pela organização.

4.6 Integração com Telegram

O bot é criado no `@BotFather`, que entrega o token. Esse token deve ser tratado como senha: qualquer pessoa que o possua controla o bot. Depois de salvo, o Notify Flow chama `getMe` para identificar nome e username. Para receber atualizações, registra uma URL HTTPS em `/api/webhooks/telegram` e envia um secret token; a API compara o cabeçalho correspondente de forma segura e deduplica `update_id` no Redis.

A pessoa precisa iniciar o bot. `/notify-me` registra autorização; `/verify-me` abre o menu de vínculo, acesso ao Meu Perfil e ajuda; `/login` entrega um link temporário. Quando o usuário compartilha o próprio contato por botão nativo, o backend confirma que

`'contact.user_id'` corresponde ao remetente antes de associar o telefone. Um e-mail digitado em qualquer ponto inicia o mesmo desafio de confirmação usado no WhatsApp.

Templates Telegram podem enviar texto, foto, vídeo e menus. Links de mídia passam por validação HTTPS, bloqueio de endereços privados para reduzir SSRF, limite de redirecionamentos, timeout, verificação de assinatura do arquivo e limites de tamanho antes de chamar `'sendPhoto'` ou `'sendVideo'`. O bot não obtém silenciosamente o número de qualquer usuário e estar no mesmo grupo não autoriza mensagem privada.

4.7 Integração com Gmail

O produto usa Nodemailer com o serviço Gmail por SMTP. A conta Gmail autentica a conexão; a senha de aplicativo, normalmente com 16 caracteres, substitui a senha principal; o campo remetente define o endereço apresentado no `'From'`, que deve ser a própria conta ou um alias autorizado. SMTP cuida do transporte entre a API e os servidores do Google, tipicamente por TLS.

Para gerar uma senha de aplicativo, a conta precisa de verificação em duas etapas. O recurso pode não aparecer em contas corporativas administradas, Advanced Protection ou configurações restritas a chaves de segurança. O Google revoga senhas de aplicativo quando a senha principal é alterada. Em uma evolução de produção com múltiplas organizações, OAuth 2.0 e a Gmail API seriam preferíveis por oferecerem escopo e revogação mais granulares; a versão atual não implementa OAuth.

4.8 Fila, tentativas, recibos e logs

Ao confirmar uma campanha, a API cria notificações e entregas no MongoDB antes de enviá-las ao BullMQ. Essa ordem evita que uma interrupção entre a requisição e o Redis apague a intenção de envio. O worker reivindica a entrega, renova um heartbeat, monta o payload próprio do canal e classifica falhas transitórias e permanentes. Itens transitórios recebem até quatro tentativas com backoff; itens sem permissão são ignorados com motivo explícito e não consomem envio.

Ciclo de entrega confiável

Persistência, fila, tentativas controladas, recibos do provedor e trilha por destinatário

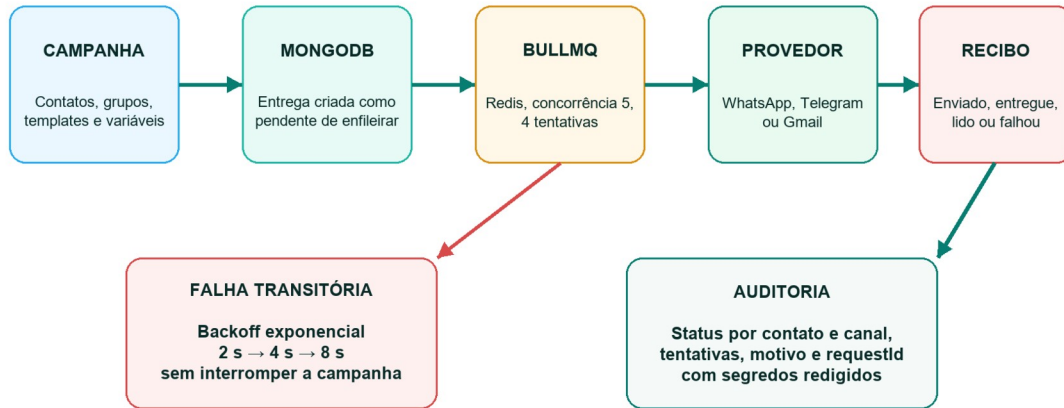


Figura 5 — Ciclo de entrega confiável, tentativas e auditoria

Fonte: Elaboração própria (2026).

O WhatsApp fornece recibos como enviado, entregue, lido ou falhou; Telegram e SMTP devolvem aceitação técnica, mas não possuem necessariamente o mesmo nível de confirmação de leitura. Por isso, os status não devem ser interpretados como equivalentes entre canais. A interface mostra o resultado de cada contato e canal, número de tentativas e mensagem operacional sanitizada. Os logs gerais expiram por padrão em 180 dias; recibos em sete dias; eventos da central e conversas Cloud em 30 dias; backups de conversas em 90 dias.

O Redis também deduplica webhooks e mantém estado efêmero. Em produção ele é obrigatório: a API falha fechada se não conseguir conectar, porque aceitar campanhas sem fila criaria falsa confiança. A política `noeviction` evita apagar trabalhos sob pressão de memória, mas exige monitoramento de capacidade.

4.9 Execução local com Docker

Na raiz do projeto, o Docker Compose cria quatro serviços: MongoDB 7, Redis 7.4, API e frontend Nginx. Apenas o frontend e a API são publicados no loopback do host; Mongo e Redis permanecem na rede interna. Os volumes `mongo\_data` e `redis\_data`

preservam dados entre recriações. A API possui health check que consulta Mongo e Redis; o frontend usa `/healthz` para verificar o Nginx.

Instalar Docker Desktop e confirmar que o daemon está ativo.

Copiar os arquivos `.env.example` adequados e preencher apenas segredos locais.

Executar `docker compose config --quiet` para validar a composição.

Executar `docker compose up --build -d` na raiz do projeto.

Abrir `http://localhost:8080` e verificar `http://localhost:8080/api/health`.

Acompanhar `docker compose logs -f api frontend` durante os testes.

Quando `MONGODB_URI` não é fornecida no Compose, a intenção é usar o serviço local. Um placeholder de Atlas copiado literalmente deve ser removido ou substituído, caso contrário a API tentará resolver um endereço inexistente. Credenciais de provedores podem ser cadastradas pela interface e ficam criptografadas no banco.

4.10 Implantação no Render por Blueprint

O arquivo `render.yaml` declara três recursos no Oregon, todos no plano `starter`: o web service `notify-flow`, o private service `api` e o Key Value `notify-flow-redis`. O frontend compila Vue e serve arquivos por Nginx. Como a API é privada, ela não possui URL pública própria; o Nginx recebe seu `hostport` pela propriedade `fromService` e encaminha `/api` e `/socket.io`. O MongoDB Atlas permanece externo e sua URI é solicitada no sincronismo do Blueprint.

O Blueprint usa `sync: false` para valores fornecidos pelo operador e `generateValue: true` para segredos internos. `PUBLIC_APP_URL` e `CORS_ORIGINS` apontam para `https://notify-flow.onrender.com/`. Se o serviço for renomeado ou receber domínio próprio, ambos devem ser atualizados, bem como os callbacks dos provedores. O webhook WhatsApp usa `https://notify-flow.onrender.com/api/webhooks/whatsapp-cloud`; o Telegram deriva sua rota pública da mesma URL.

Domínio	Variáveis exemplares	Regra de produção
Aplicação	NODE_ENV, PORT, API_PREFIX, PUBLIC_APP_URL	URLs HTTPS e prefixos coerentes com o proxy.
Dados	MONGODB_URI, REDIS_URL, REDIS_REQUIRED	Atlas e Key Value privados; nunca versionar URI real.
Autenticação	JWT_*_SECRET, PROFILE_JWT_SECRET, TTLs	Segredos independentes e fortes; senha admin ≥ 12.

Domínio	Variáveis exemplares	Regra de produção
Proteção	ENCRYPTION_KEY, SEARCH_HASH_KEY, RATE_LIMIT_*	Chaves distintas, proxy confiável e limites ativos.
Canais	WHATSAPP_*_VERSION, comandos, webhook secret	Credenciais preferencialmente pela UI criptografada.

Quadro 5 — Variáveis de ambiente por domínio

Fonte: Elaboração própria (2026).

A rota pública `/healthz`` prova que o Nginx está de pé, enquanto `/api/health`` atravessa o proxy e verifica API, Mongo e Redis. O Blueprint atual não declara health check próprio para o private service; portanto, recomenda-se monitorar externamente `/api/health`` e adicionar uma verificação equivalente à API em uma revisão futura.

4.11 Whitelabel, mídia e administração do armazenamento

A instalação pode alterar nome, título da página, logo, cores, rodapé e links institucionais sem modificar o código. Trata-se de whitelabel por instalação, e não de um modelo SaaS multitenant. A mesma tela apresenta o consumo do MongoDB por collection e total, com ajuda contextual para explicar os domínios armazenados.

Imagens, vídeos e documentos vinculados a templates são persistidos no GridFS. O upload inspeciona tipo real, extensão, tamanho e magic bytes; a API publica uma capability HTTPS assinada para que a Meta obtenha o conteúdo. Rascunhos expiram, arquivos referenciados são retidos e a remoção lógica invalida imediatamente a URL antes da coleta física. Esse ciclo evita base64 em documentos BSON e reduz arquivos órfãos.

O administrador pode exportar uma collection ou o conjunto permitido em JSON sanitizado e ZIP com manifesto/mídias, respeitando limites de volume. A exportação remove segredos, hashes e campos criptografados; portanto, é evidência e portabilidade parcial, não backup restaurável. A limpeza exige confirmação forte, bloqueia operações conflitantes, coordena a fila e registra auditoria com horário, ator, escopo e resultado. Collections de autenticação e configuração possuem proteções adicionais.

4.12 Webhooks, conversas, alertas e tempo real

O webhook WhatsApp valida o challenge de configuração e a assinatura HMAC do corpo bruto. Cada evento é criptografado, deduplicado e processado por lease, inclusive payloads técnicos ou tipos não suportados que precisam permanecer visíveis ao operador sem criar contato automaticamente. A aba Webhook separa eventos, contatos processados e falhas da área de disparos e conversas.

O Telegram valida o secret token, deduplica update_id e encaminha comandos, callbacks, contatos compartilhados e grupos vinculados. As conversas dos dois canais carregam as dez mensagens mais recentes e paginam para trás ao rolar, preservando foco no conteúdo novo. A interface administrativa recebe mensagens, receipts e logs por Socket.IO autenticado.

Opcionalmente, uma conta Gmail completa pode cadastrar um endereço operacional para receber alertas de mensagens novas. A API persiste antes do ACK uma outbox criptografada e idempotente, processada por worker com lease, recuperação de tarefas obsoletas e até três tentativas. O assunto identifica o canal, o contato e um trecho seguro; falha SMTP nunca impede o provedor de receber a resposta do webhook.

5 VALIDAÇÃO E RESULTADOS FINAIS

5.1 Ambiente, procedimento e evidências

A validação final foi executada em 14 de agosto de 2026 sobre a branch principal e combinou suítes automatizadas, lint, build, inspeção de infraestrutura e evidências funcionais. Os totais abaixo foram obtidos pela execução efetiva dos comandos, não por estimativa de arquivos. O caminho canônico do workspace foi usado no frontend para evitar uma particularidade local de junction do Windows; isso não representa defeito do produto.

Evidência	Escopo	Resultado observado
Testes da API	40 arquivos: arquitetura, filas, webhooks, segurança, storage e canais.	374 aprovados; 0 falhas.
Testes do frontend	35 arquivos: sessão, páginas, dialogs, canais, whitelabel e previews.	244 aprovados; 0 falhas.
Total automatizado	API + frontend.	618 aprovados; 0 falhas.
Lint da API	ESLint com zero warning permitido.	Aprovado.
Build do frontend	Vite em modo de produção.	Aprovado.
Infraestrutura	Compose, Dockerfiles, Nginx e Render Blueprint.	Revisada e documentada.

Tabela 2 — Evidências de testes e validações finais

Fonte: Elaboração própria (2026).

O total evoluiu de 514 verificações registradas no Gate 4 para 618 no Gate 5. Os novos casos abrangem whitelabel, lifecycle de mídia, armazenamento, exportação/limpeza, templates dinâmicos, eventos técnicos de webhook, conjuntos e alertas por email. A quantidade não representa percentual de cobertura; sua função é demonstrar amplitude temática e regressão controlada.

5.2 Ensaio de envio em massa e integrações

O autor realizou um ensaio funcional com aproximadamente 20 a 30 usuários simultaneamente. As campanhas foram processadas em fila; contatos sem autorização foram ignorados com motivo, falhas de provedor permaneceram isoladas e os demais

envios continuaram. Os logs registraram status por contato e canal de forma coerente. A Meta, o Telegram e o Gmail foram exercitados com credenciais de teste/produção controladas durante a implementação.

Limite da evidência

O ensaio observado não mede throughput máximo, latência percentil, capacidade sustentada ou SLA. Para essas conclusões seriam necessários gerador de carga, ambiente estabilizado, massa representativa, repetição e análise estatística.

5.3 Resultados por requisito e impacto

O resultado técnico mais relevante é o isolamento de falhas. Em uma campanha com três canais, um contato pode não ter Telegram, outro pode ter email revogado e um modelo WhatsApp pode ser rejeitado. Cada entrega preserva estado, tentativas e motivo próprios; o lote não transforma essas ocorrências em uma falha única.

O segundo resultado é a rastreabilidade do consentimento. Interação, comando, convite, decisão administrativa e revogação são distinguíveis. O titular consulta seus dados, grupos, convites e entregas, enquanto o administrador dispõe de evidência operacional sem visualizar segredos.

No WhatsApp, a Cloud API oficial reduz o risco inerente a automações de sessão pessoal e oferece modelos, limites e receipts suportados. A plataforma não promete imunidade a bloqueios: qualidade, opt-out, categoria, cobrança e políticas da Meta continuam soberanos.

Risco ou limitação	Controle atual	Evolução recomendada
Indisponibilidade externa	Fila, retries e falha por entrega.	Circuit breaker e métricas de disponibilidade.
Semântica distribuída	Idempotência, claim, lease e deduplicação.	Reconciliação ampliada; não prometer exactly-once.
Escala horizontal	Worker e Socket.IO na mesma API.	Separar workers e adotar adapter Redis.
Exportação sanitizada	Remove segredos e PII cifrada.	Backup externo/PITR e teste de restauração.
Usabilidade	Fluxos responsivos e testes funcionais.	Estudo com usuários e métricas de tarefa.
Deploy sem CI formal	Comandos reproduzíveis e auto deploy.	Pipeline CI antes do deploy automático.
Conformidade	Consentimento, direitos e segurança técnica.	Governança institucional e validação jurídica.

Tabela 3 — Riscos, controles e limitações

Fonte: Elaboração própria (2026).

6 ENTREGA, OPERAÇÃO E MANUTENÇÃO

6.1 Manifesto do pacote Gate 5

A entrega reúne código da API e do frontend, arquivos de lock, Dockerfiles, Docker Compose, Render Blueprint, exemplos de ambiente sem segredos, documentação Markdown, relatório DOCX/PDF e diretório reservado para mídias de evidência. Slides não integram esta solicitação. O README da raiz funciona como sumário e direciona para arquitetura, implantação, testes, segurança, canais e checklist.

O repositório canônico informado para depósito é `'https://github.com/samuelvictorol/tcc-prti-samuelvictor'`. A variante `'-mvp'` foi a base ativa de desenvolvimento durante parte do trabalho e deverá ser sincronizada manualmente pelo autor. A documentação preserva essa linhagem sem tratar a base intermediária como destino final.

6.2 Execução e verificação operacional

copiar ``.env.example`` para ``.env`` e substituir todos os placeholders por segredos exclusivos;
executar ``docker compose --env-file .env.example config --quiet`` antes de subir o ambiente;
executar ``docker compose up --build -d`` e acompanhar os health checks;
validar ``/healthz`` e ``/api/health`` antes de configurar os provedores;
cadastrar e testar um canal por vez; depois validar convite, consentimento e campanha;
verificar fila, receipts, webhook, logs, opt-out e exportação sanitizada;
documentar mudanças de domínio em `PUBLIC_APP_URL`, `CORS` e callbacks externos.

6.3 Empacotamento seguro e retenção

A raiz local não deve ser compactada diretamente porque pode conter ``.env``, ``.git``, caches, dependências e builds. O script ``scripts/package-gate5.ps1`` cria uma pasta de estágio limpa e exclui segredos, ``node_modules``, ``dist``, logs e temporários. O arquivo gerado deve passar por revisão manual antes do depósito. Não se deve incluir dumps reais, tokens, URLs assinadas temporárias ou exports com pessoas identificáveis.

A política operacional distingue retenção funcional de backup. Logs e eventos usam TTLs; conversas locais e alertas administrativos possuem ciclos definidos; mídias órfãs são coletadas; e exports administrativos são sanitizados. Recuperação de desastre requer

backup gerenciado do MongoDB/Atlas e ensaio de restauração fora do próprio banco da aplicação.

6.4 Roteiro de aceite do produto

Demonstração técnica sugerida

1) conferir saúde e whitelabel; 2) mostrar credenciais mascaradas; 3) abrir convite; 4) autorizar um contato; 5) criar/clonar templates e revisar um conjunto; 6) enfileirar campanha; 7) acompanhar deliveries e webhook; 8) responder em conversa; 9) abrir Meu Perfil; 10) mostrar storage e auditoria.

7 CONSIDERAÇÕES FINAIS

O Gate 5 consolida o Notify Flow como produto técnico final demonstrável na modalidade Projeto de Aplicação. A solução integra três canais, oferece interfaces administrativas e públicas, processa campanhas em fila, recebe eventos por webhook e atualiza o painel em tempo real. As 618 verificações automatizadas aprovadas, o lint, o build e a revisão da infraestrutura sustentam a conclusão técnica sem implicar ausência absoluta de defeitos.

A contribuição central é organizar uma comunicação que começa no consentimento e termina na auditoria. Para WhatsApp, isso significa trabalhar com a Cloud API, a janela de atendimento e os modelos aprovados; para Telegram, reconhecer a identidade observada pelo bot; para e-mail, validar o endereço. A fila impede que uma falha parcial apague o resultado da campanha e os recibos deixam clara a diferença entre envio, aceitação, entrega e leitura.

Como continuidade, recomenda-se submeter os fluxos a usuários externos, medir taxa de conclusão, tempo de tarefa e compreensão de consentimento; adicionar CI; separar o worker da API; adotar adapter Redis do Socket.IO; fortalecer retenção e backup; avaliar OAuth para Gmail; e implementar monitoramento sintético do ambiente Render. Uma distribuição desktop com Electron permanece como evolução posterior e não integra o produto técnico do Gate 5.

Conclui-se que os objetivos definidos foram alcançados no escopo declarado: o produto oferece mensageria oficial, consentimento independente, filas recuperáveis, auditoria, autogestão do contato, configuração whitelabel e implantação reproduzível. Seu impacto esperado é reduzir o uso inadequado de integrações não oficiais, aumentar a rastreabilidade operacional e devolver ao titular controle explícito sobre os canais pelos quais aceita ser notificado.

O resultado deve ser interpretado com prudência: o Notify Flow não garante entrega, não substitui políticas dos provedores, não certifica conformidade jurídica e não foi submetido a benchmark formal ou pentest externo. Essas limitações representam fielmente o estado do software.

REFERÊNCIAS

BRASIL. Lei nº 13.709, de 14 de agosto de 2018. Lei Geral de Proteção de Dados Pessoais (LGPD). Brasília, DF, 2018.

BULLMQ. Retrying failing jobs. Disponível em: <https://docs.bullmq.io/guide/retrying-failing-jobs>. Acesso em: 14 ago. 2026.

BULLMQ. Workers — concurrency. Disponível em: <https://docs.bullmq.io/guide/workers/concurrency>. Acesso em: 14 ago. 2026.

DOCKER. Docker Compose documentation. Disponível em: <https://docs.docker.com/compose/>. Acesso em: 14 ago. 2026.

GOOGLE. Sign in with app passwords. Disponível em: <https://support.google.com/accounts/answer/185833>. Acesso em: 14 ago. 2026.

GOOGLE. Implement server-side authorization — Gmail API. Disponível em: <https://developers.google.com/workspace/gmail/api/auth/web-server>. Acesso em: 14 ago. 2026.

META. WhatsApp Business Platform pricing. Disponível em: <https://whatsappbusiness.com/products/platform-pricing/>. Acesso em: 14 ago. 2026.

META. WhatsApp Business Platform onboarding guide. Disponível em: <https://whatsappbusiness.com/resources/resource-library/api-onboarding/>. Acesso em: 14 ago. 2026.

META. Documents for business verification. Disponível em: <https://www.facebook.com/help/243868559497297/>. Acesso em: 14 ago. 2026.

META. WhatsApp Cloud API error codes. Disponível em: <https://developers.facebook.com/docs/whatsapp/cloud-api/support/error-codes>. Acesso em: 14 ago. 2026.

MONGODB. GridFS specification. Disponível em: <https://www.mongodb.com/docs/manual/core/gridfs/>. Acesso em: 14 ago. 2026.

RENDER. Blueprint YAML reference. Disponível em: <https://render.com/docs/blueprint-spec>. Acesso em: 14 ago. 2026.

RENDER. Private services. Disponível em: <https://render.com/docs/private-services>. Acesso em: 14 ago. 2026.

RENDER. Key Value. Disponível em: <https://render.com/docs/key-value>. Acesso em: 14 ago. 2026.

TELEGRAM. Telegram Bot API. Disponível em: <https://core.telegram.org/bots/api>. Acesso em: 14 ago. 2026.

APÊNDICE A — HISTÓRIAS DE USUÁRIO

ID	História	Critério resumido
A01	Como administrador, quero configurar cada canal separadamente para implantar o sistema por etapas.	Um canal incompleto não bloqueia os demais.
A02	Como administrador, quero identificar novas interações em tempo real.	Evento aparece sem recarregar e sem revelar segredo.
A03	Como administrador, quero cadastrar templates por formulário.	Não é necessário editar JSON.
A04	Como administrador, quero agrupar templates por campanha.	Conjunto possui até um template por canal.
A05	Como administrador, quero selecionar contatos e grupos.	Somente destinos identificados e permitidos são enviados.
A06	Como administrador, quero revisar cada mensagem antes de enviar.	Preview responsivo precede a confirmação.
A07	Como administrador, quero acompanhar cada contato e canal.	Status, tentativas e erro ficam detalhados.
A08	Como administrador, quero sincronizar grupos por convite.	Novos contatos autorizados são incluídos sem duplicidade.
A09	Como administrador, quero responder no WhatsApp.	Texto livre só é permitido na janela de 24 horas.
A10	Como administrador, quero versionar termos e privacidade.	Documento vigente é apresentado no convite.
A11	Como administrador, quero auditar links de acesso.	Uso, validade e revogação são registrados.
A12	Como administrador, quero que uma falha não interrompa a campanha.	Cada entrega é processada e registrada isoladamente.
A13	Como administrador, quero personalizar a identidade visual da instalação.	Nome, logo, cores, rodapé e links institucionais são aplicados sem alterar o código.
A14	Como administrador, quero medir e administrar o armazenamento.	Uso por collection, exportação sanitizada, limpeza confirmada e auditoria ficam disponíveis.
A15	Como administrador, quero receber por e-mail um resumo das mensagens recebidas.	A configuração é opcional, depende do Gmail e usa uma outbox idempotente com tentativas.

Tabela 4 — Histórias de usuário do administrador

Fonte: Elaboração própria (2026).

ID	História	Critério resumido
U01	Como contato, quero escolher quais canais autorizo.	Permissões independentes e revogáveis.

ID	História	Critério resumido
U02	Como contato, quero iniciar a autorização pelo canal.	Comando ou convite resulta em identidade observada.
U03	Como contato, quero vincular Telegram e WhatsApp com segurança.	Telefone só é aceito quando compartilhado pelo próprio remetente.
U04	Como contato, quero validar meu e-mail no chat.	Código expira em 15 min e reenvio respeita 2 min.
U05	Como contato, quero entrar no Meu Perfil sem senha permanente.	Link assinado, temporário e de uso único.
U06	Como contato, quero consultar minhas entregas.	Histórico é restrito ao próprio contato.
U07	Como contato, quero compreender convites e grupos.	Perfil lista vínculos e participantes mascarados.
U08	Como contato, quero remover meu vínculo.	Ação afeta somente o próprio cadastro e exige confirmação.
U09	Como contato, quero revogar um canal.	Novos envios são bloqueados e a decisão fica auditada.
U10	Como visitante, quero uma página de convite responsiva.	Links e QR funcionam em desktop e celular.
U11	Como contato, quero descobrir os comandos disponíveis no próprio canal.	O comando /help apresenta as ações suportadas sem exigir acesso administrativo.

Tabela 5 — Histórias de usuário do contato

Fonte: Elaboração própria (2026).

APÊNDICE B — CHECKLIST DE CONFIGURAÇÃO

B.1 WhatsApp Cloud

- Confirmar empresa, Página, portfólio e verificações de segurança.
- Registrar número elegível e associar forma de pagamento.
- Criar Meta App, adicionar o produto WhatsApp e selecionar a WABA.
- Obter Phone Number ID, Business Account ID e token apropriado.
- Cadastrar App Secret e verify token no Notify Flow.
- Configurar callback HTTPS `/api/webhooks/whatsapp-cloud` e assinar `messages`.
- Criar modelos no WhatsApp Manager e aguardar aprovação.
- Cadastrar no Notify Flow o mesmo nome, idioma e parâmetros.
- Testar número de teste e, depois, número de produção com destinatário autorizado.

B.2 Telegram

- Criar o bot no @BotFather e armazenar o token como segredo.
- Salvar o token; conferir nome e username identificados por getMe.
- Garantir URL pública HTTPS e registrar webhook com secret token.
- Iniciar o bot, testar /notify-me, /verify-me e /login.
- Compartilhar o próprio contato para validar o vínculo de telefone.
- Testar texto, foto/vídeo por URL segura e menu de botões.

B.3 Gmail

- Ativar verificação em duas etapas na Conta Google.
- Gerar senha de aplicativo quando o recurso estiver disponível.
- Cadastrar conta Gmail, senha de aplicativo e remetente autorizado.
- Opcionalmente, cadastrar o destinatário operacional dos alertas de mensagens recebidas.
- Enviar teste individual e campanha com endereço autorizado.
- Confirmar que falha SMTP de um contato não interrompe os demais.

B.4 Produção

- Criar MongoDB Atlas com usuário de menor privilégio, rede e backup/PITR.
- Sincronizar o Blueprint e fornecer MONGODB_URI e credenciais administrativas.
- Verificar que frontend, API privada e Key Value estão na mesma região.
- Testar `/healthz` e `/api/health`.
- Atualizar callbacks ao alterar domínio.
- Executar smoke test de convite, consentimento, campanha e Meu Perfil.
- Monitorar memória do Key Value, fila, erros por provedor e qualidade da WABA.

APÊNDICE C — MATRIZ DE EVIDÊNCIAS

Área	Evidência no projeto	Validação do Gate 5
API	`api/src`, 22 modelos lógicos, managers, serviços, DTOs e 40 arquivos de teste.	374 testes e lint aprovados.
Frontend	14 rotas funcionais, 35 arquivos de teste e build Vue/Quasar/Vite.	244 testes e build aprovados.
Fila	BullMQ, Redis obrigatório, Mongo durável e recovery sweep.	Retries e falha parcial cobertos por testes.
Webhooks	HMAC WhatsApp, secret Telegram, eventos persistidos e deduplicação.	Contratos, tipos não suportados e telas de inspeção testados.
Mídia	GridFS, URLs assinadas, validação de tipo, lifecycle e proteção SSRF.	Upload, entrega, tombstone e limpeza cobertos por testes.
Whitelabel	Identidade visual, logo, paleta, rodapé e links por instalação.	Persistência, validação e aplicação visual testadas.
Armazenamento	Uso por collection, exportação sanitizada, limpeza forte e auditoria.	Contratos, confirmação e limites testados.
Infra local	`docker-compose.yml` com quatro serviços.	Compose validado estaticamente.
Infra produção	`render.yaml`: web, private service e Key Value.	Blueprint inspecionado e deploy disponível.
Governança	ConsentEvent, termos, Meu Perfil e pseudonimização.	Fluxos e políticas cobertos por testes.

Tabela 6 — Matriz de rastreabilidade dos requisitos finais

Fonte: Elaboração própria (2026).

APÊNDICE D — COMANDOS DE VALIDAÇÃO E EMPACOTAMENTO

Os comandos abaixo permitem repetir a verificação final sem depender de estado oculto. Credenciais reais devem permanecer apenas no arquivo `.env` local e nos painéis dos provedores.

API: `npm run check`

Frontend: `npm test -- --run --reporter=dot`

Build: `npm run build`

Infraestrutura: `docker compose config --quiet`

Execução local: `docker compose up --build -d`

Pacote sanitizado: `powershell -ExecutionPolicy Bypass -File scripts/package-gate5.ps1 -CreateZip`

APÊNDICE E — ORGANIZAÇÃO DAS EVIDÊNCIAS

Capturas e vídeos devem ser armazenados em docs/app-media/screenshots e docs/app-media/videos. Cada arquivo deve indicar data, jornada e resultado, com telefones, e-mails, tokens, IDs de provedor, URLs assinadas e demais dados pessoais mascarados. O diretório possui um README com a convenção de nomenclatura e um roteiro mínimo de evidências para banca e reprodução.

APÊNDICE F — EVOLUÇÃO DOCUMENTAL

O Gate 4 permanece como linha de base histórica de impacto, testes e validação. O presente relatório substitui seus números de teste e amplia a descrição do produto com conjuntos de templates, conversas, eventos de webhook, mídia em GridFS, whitelabel, gestão de armazenamento, alertas por Gmail e empacotamento seguro. O README da raiz e docs/README.md atuam como índices para a documentação técnica detalhada do Gate 5.